

Who Finishes the Job? A Study of Follow-Up Fixes and Commit Authorship on AI Coding Agent Pull Requests

Wannita Takerngsaksiri · Nhat Duong ·
Scott Barnett

Received: date / Accepted: date

Abstract AI coding agents now author a large share of pull requests (PRs) merged into popular open-source projects. A merged agent PR is usually considered finished work; yet, prior studies have reported issues in agent code after the merge (e.g., code smells and static-analysis issues). However, little is known about how often a merged agent PR is fixed afterward, and who actually authors the fixing. In this paper, we follow 6,774 merged agent PRs across five AI coding agents (OpenAI Codex, GitHub Copilot, Devin, Cursor, and Claude Code) from the AIDev-pop dataset (open-source repositories with at least 500 stars) into their follow-up fixes, against a baseline of 5,044 contemporaneous human PRs from the same repositories. We link each merge to its candidate fixes, verify every candidate with human annotators and an LLM judge that matches human-level agreement (binary Cohen’s $\kappa = 0.78$ against a human–human $\kappa = 0.77$, Direct-fix precision 90%), and attribute the fixing work at the PR and the commit level. Our findings show that (1) merged agent PRs attract verified fixes at 1.62 times the odds of merged human PRs in the same repositories over the same period of time; (2) 69.6% of verified fixes in agent merges come from the same agent; and (3) 76.4% of the verified fix PRs are agent-authored throughout all commits. These results show that agents currently largely finish their own job, but their merges still require fixing more often than human merges.

Keywords AI coding agents · Human-agent collaboration · Empirical study · Mining software repositories · Pull requests · Human validation

W. Takerngsaksiri · N. Duong · S. Barnett
Applied AI Initiatives (A2I2), Deakin University, Melbourne, Australia
E-mail: wannita.takerngsaksiri@deakin.edu.au

1 Introduction

AI coding agents (e.g., Codex (OpenAI, 2026), Devin (Cognition, 2026), Copilot (GitHub, 2026), Cursor (Anysphere, 2026), and Claude Code (Anthropic, 2026)) have revolutionised software engineering at a remarkable speed, transforming from assisting as auto-completion to taking authorship. Agents now open, revise and merge pull requests (PRs) in their own workflows, on the scale of hundreds of thousands of repositories in both open source and industry (Li et al., 2026; Robbes et al., 2026; Takerngsaksiri et al., 2025).

However, the review an agent PR passes before merging is quick and light: 61.4% of agent PRs do not receive recorded human review (Duma et al., 2026), and agent PRs complete within minutes to hours (Watanabe et al., 2026). Previous studies have also reported problems in agent code after the merge (e.g., static-analysis issues that persist (Liu et al., 2026), and later modifications that are corrective rather than adaptive (Rahman and Shihab, 2026; Khemissi et al., 2026)). As a result, when a merged change contains an issue, the consequence is a *follow-up fix*: a later PR that revisits and corrects the merged PR. Understanding this follow-up fix caused by agents can help developers plan their tasks and how agent contributions should be evaluated. Nevertheless, little is known about these follow-up fixes on merged agent PRs. Specifically, how *often* merged agent PRs create follow-up fixes, *who* comes back to fix them, and *how* merged PRs with and without a follow-up fix differ have not been measured.

To address this gap, we follow 6,774 merged agent PRs across 891 repositories and five AI coding agents in AIDev-pop dataset into their follow-up fixes. We analyse *how often* follow-up fixes occur, *who* authors those fixes, and *how* PRs with and without a follow-up fix differ, against a contemporaneous baseline of 5,044 merged human PRs from the same repositories. We link each merge to its candidate follow-up fixes and verify every candidate with human annotators and a validated LLM judge. Then, we classify who opened each fixing PR and who authored each of its commits. Finally, we address the following four research questions:

RQ1. How often are the merged agent PRs followed by a fix compared to the merged human PRs in the same repositories?

Result. 22.9% of merged agent PRs are followed by a candidate fix and 4.5% by a verified fix within 30 days. In the same repositories and the same observation window, agent merges are fixed at 1.41 times the human odds (candidate) and 1.62 times (verified), and the fixes arrive early: half of the 30-day incidence accumulates within the first week.

RQ2. Who authors the verified fix PRs of the merged agent PRs?

Result. 69.6% of the verified fixes on agent merges come from the same agent, while 89% of the verified fixes on human merges also come from humans. The finding shows that fixing work stays within the authoring population on both sides.

RQ3. To what extent are the commits pushed to the verified fix PRs authored by agents?

Result. The commits added to verified-fix agent PRs between first push and merge are, on average, 87.4% agent-authored per PR, and 76.4% of those PRs are agent-authored throughout all commits.

RQ4. How do merge-time signals differ between merged agent PRs that are followed by a fix and those that are not?

Result. Overall, merge-time signal distributions of agent PRs on both groups are nearly identical (all effects negligible to small). However, within repositories, the number of commits to complete a PR could mark a follow-up fix. Specifically, a merge with ten times the commits carries 6.1 times the odds of a verified fix.

This paper makes the following contributions:

1. We conduct an empirical study on the authorship of follow-up fixes of merged agent PRs at scale, evaluating against a human baseline in the same repositories over the same period of time.
2. We perform a human validation to verify candidate fixes in agent PRs and an LLM judge that matches human-level agreement (Cohen’s $\kappa = 0.78$, Direct-fix precision 90%) to ensure the relevance throughout the dataset.
3. We perform a comprehensive analysis of the differences in merge-time signals between agent PRs with and without a follow-up fix.
4. Our replication package is released at <https://github.com/wannita901/pr-fix-authorship>.

The remainder of the paper proceeds as follows: Section 2 positions our paper with other related works, Section 3 presents the study design, Section 4 reports each RQ’s approach and results, Section 5 discusses our findings, Section 6 reports threats to validity, and Section 7 concludes our study.

2 Background and Related Work

2.1 AI Coding Agents in Software Development

AI coding agents (e.g., Codex (OpenAI, 2026), Devin (Cognition, 2026), Copilot (GitHub, 2026), Cursor (Anysphere, 2026), and Claude Code (Anthropic, 2026)) have become routine contributors to open-source projects (Robbes et al., 2026; Li et al., 2025) and industry (Takerngsaksiri et al., 2025). Robbes et al. (2026) studies more than 120,000 repositories and estimates the adoption rate of 22.20%–28.66% in February 2026. Li et al. (2025) describe this shift as the move from *SE 2.0* (human-authored, agent-assisted) to *SE 3.0* (agent-authored, human-reviewed), and release AIDev (Li et al., 2025), the dataset on which we build our analysis. Nearly a million agent pull requests are distributed in more than a hundred thousand popular repositories open sources on GitHub in 2025. On the other hand, Takerngsaksiri et al. (2025) conducted a study of the AI coding agent with human-in-the-loop, showing how agents were adopted at Atlassian, piloting over 600 real-world JIRA issues in production in a month.

Therefore, while the adoption rate of AI coding agents is increasing rapidly, this highlights the importance of our study in identifying what is left behind after an agent PR merges into production and who actually follow up to fix those issues *operationally*.

2.2 The Study of AI Coding Agent’s Pull Requests

Pull requests of AI coding agents have been a popular research topic in recent years. Li et al. (2026) present AIDev dataset, which is a large-scale collection of agent commits and pull requests crawled from GitHub in 2025. Subsequent works have grown rapidly around it in various angles: from studying at *first-push characteristics* (Ogenrwot and Businge, 2026; Watanabe et al., 2026; Gong et al., 2026), *review process* (Duma et al., 2026; Haider and Zimmermann, 2026), *merge outcomes* (Peralta et al., 2026; Alam et al., 2026), to *after the merge* (Liu et al., 2026; Rahman and Shihab, 2026; Khemissi et al., 2026).

Starting at the *first push* of an agent PR, previous studies analyse the characteristics of agent’s signals compared to human PRs. The studies found that agent PRs are smaller and more localised than human PRs (Ogenrwot and Businge, 2026), differ in description style (Watanabe et al., 2026), and occasionally misdescribe their own diff (Gong et al., 2026).

Next, during the *code review process*, there are studies on the characteristics of review comments on AI agent PRs. Duma et al. (2026) found that 61.4% of agent PR do not receive human review records, while Haider and Zimmermann (2026) found that the most common themes of comments on agent PRs are functional correctness.

Then, there are the *merge outcomes* studies in which the merged and unmerged agent PRs have been analysed to find differences. Peralta et al. (2026) found that unmerged outcomes substantially overstate agent error with only 35.7% of rejected PRs reflected clear agent failures. Additionally, Alam et al. (2026) indicate test case failures and prior resolution of the same issues by other PRs are the most common causes of an unmerged PR.

Last but not least, there are studies that involve activities *past the merge*, which our paper also falls into this category. Liu et al. (2026) found that 22.7% of the static-analysis issues introduced by agents still survive to the latest version of the repositories at the time. Rahman and Shihab (2026) conducted survival analysis on agent code and found that agent code is modified *less* often than human code, though its modifications are corrective more often than adaptive, unlike human code. Similar to previous findings, Khemissi et al. (2026) mined the follow-up PRs that *explicitly reference* an agent PR and found that more than half of agent-to-agent references are for corrective.

Different to prior works, we mine the *follow-up fixes* of merged agent PRs, and verify the fix candidates with human validation and an LLM judge that aligns with human agreement. Our goal is to study how often agent PRs receive a follow-up fix compared to a human baseline under the same conditions.

2.3 Code Ownership in the Age of AI Coding Agents

Code ownership research traditionally links who authors a component to its quality. Bird et al. (2011) find that the number of *minor* contributors to a Windows binary correlates with its failures more strongly than size, churn, or complexity. The study indicates that changes being routed to the component’s owner rather than made by outsiders could minimise defects. On the other hand, in open-source projects, the relationship is more nuanced. Although the correlation between ownership and software quality is confirmed, ownership metrics add little beyond other measures (Foucault et al., 2015).

In the age of AI coding agents, the question of code ownership has reopened. Developers’ felt ownership of code declines as agent autonomy increases (Seo et al., 2026), while the agents’ terms of service consistently assign legal responsibility for the generated code to the human user (Treude, 2026). Sawada et al. (2026) randomly tracked 508 agent-generated files and find that humans author 83.2% of all commits, indicating a large human contribution over agent code. However, little is known about follow-up fix after a PR merges and who actually authors them. Before the agents era, studies of self-fixing report that developers fix roughly half of their own defects at individual granularity: 55.7% of single-statement bugs (Zhu and Godfrey, 2021), 46% of 8,033 fuzzer-detected bugs (Keller et al., 2023), and about half of fixed technical-debt issues (Tan et al., 2022), with a sense of responsibility for one’s own code as the main reported motivation (Tan et al., 2021).

Different to prior work, we measure the *authorship* of a *follow-up fix* after the agent PRs have been merged and whether the author class that merged a PR is the one that returns to fix it.

3 Study Design

3.1 Overview

Figure 1 shows the overview of our study. Using merged agent PRs and merged human PRs as a baseline in the same repositories, we find the follow-up fixes in RQ1. Given PR P as a merged PR and PR Q as a follow-up fix PR, we find the relationship between $P \rightarrow Q$ by defining *candidate* and *verified* fix. First, we find the *candidate* fixes according to the heuristic criteria. Then, we perform human validation and LLM judge to confirm the pairs as *verified* fixes. The verified fixes feed RQ2, which classifies who *opened* each fixing PR (the same agent, another agent, or a human). RQ3 zooms from PRs into commits which the author-class classifier labels every commit. Finally, RQ4 compares the merges that received a follow-up fix against those that did not, on four signals visible at merge time. Each RQ’s detailed approach is described with its results in Section 4.

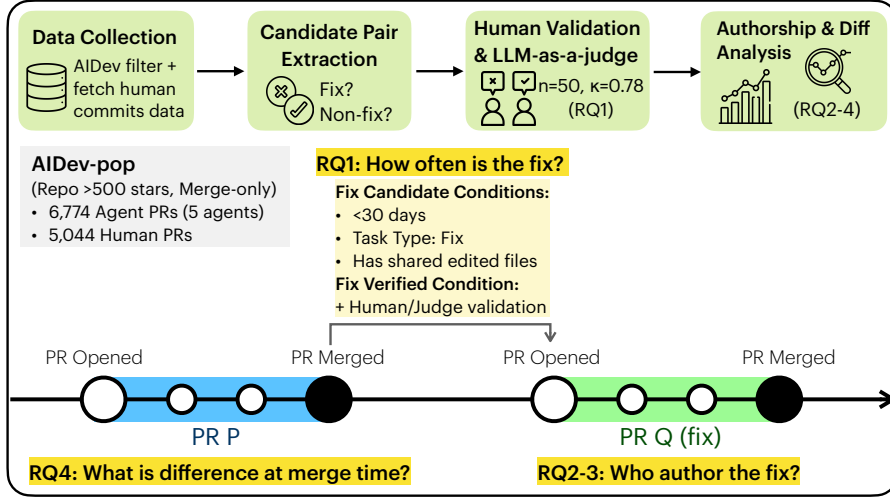


Fig. 1 The overview of our study design.

3.2 Dataset

AIDev (Li et al., 2026, 2025) is a large GitHub pull request dataset of AI coding agents with more than 450,000 pull requests and 60,000 open-source repositories. In this paper, we use AIDev-pop, a filter cohort of popular repositories in AIDev. We follow the original filter of AIDev-pop (Li et al., 2025), filtering popular repositories with more than 500 GitHub stars as of June 22, 2025. Then, we restricted our filter to only merged PRs.

Our data yields 6,774 merged agent PRs across 891 repositories, covering five agents: OpenAI Codex (2,834), Devin (1,813), GitHub Copilot (1,428), Cursor (569), and Claude Code (130). The agent PRs span December 24, 2024, to July 30, 2025. The human PRs are AIDev’s curated `human.pull.request` table with the same ≥ 500 -star repositories and merge-only conditions, resulting in 5,044 human PRs spanning January 1 to June 28, 2025. The original AIDev’s human PRs carry PR-level columns only; therefore, we re-fetch their commits, diffs, reviews, and comments per PR via the GitHub REST API. In the cross-cohort comparison, we use the human cohort’s end date as the shared cut-off for the same observation window. All analyses are restricted to merged PRs unless otherwise stated.

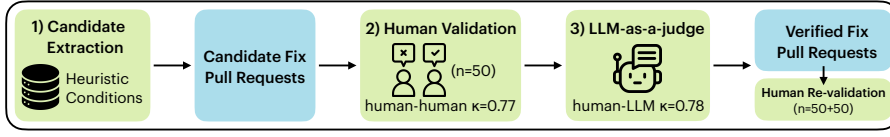


Fig. 2 (RQ1) The overview of the steps to identify verified fix PRs. A green box is a process; a blue box is a PR cohort.

4 Results

4.1 RQ1: How often are the merged agent PRs followed by a fix compared to the merged human PRs in the same repositories?

4.1.1 Approach of RQ1

We identify verified fix PRs in three main steps: 1) candidate extraction, 2) human validation, and 3) LLM-as-a-judge. The overview is shown in Figure 2.

1) Candidate Extraction. Given a merged PR P , our goal is to find a later *candidate* Q that fixes PR P . To do so, we apply four heuristic filters in order on both agent P and human P cohorts:

1. Q merges in the same repository, strictly after P , within 30 days.
2. Q is classified as `fix` type by the task-type tag in the AIDev dataset.
3. Q co-edits at least one file that P merged, identified by the intersection of net base-to-merge diff in both P and Q .
4. Q co-edited files include at least one non-boilerplate file. This is because files such as `lockfiles`, `changelogs`, and generated artifacts churn in nearly every PR.

Candidate fixers Q are drawn from both the agent and the human cohorts, as an issue could also be fixed by a human or an agent. Additionally, since Q requires a 30-day observation period, we drop every partially observed merge P . Specifically, P is excluded if P merged less than 30 days before the dataset end date. The complete details can be found in the replication package.

2) Human Validation We perform a human validation on the *candidate* fix pairs of agent PRs to identify the verified fixes. Two annotators (the first and second author with 3-4 years of coding experience) independently label a 50-pair sample through a web labeler (see Appendix A). The PR information shown on a label page is: titles, descriptions, commit messages, and shared-file net diffs. Each candidate pair is labeled as *Direct fix* (Q fixes the specific change P introduced), *Related touch*, or *Unrelated*. We consider a pair as a *verified fix* only when labeled *Direct fix*. On the binary verified-fix condition (*Direct fix* vs. *non-fix*), the two human annotators agree on 45 of the 50 sample pairs (90% agreement, Cohen’s $\kappa = 0.77$ indicating substantial agreement (Cohen, 1960; Landis and Koch, 1977)).

3) LLM-as-a-judge An LLM judge (Claude Opus 4.8, temperature=0) receives the same PR information as human annotators to label the full candidate population: 2,510 agent-merge pairs and 2,206 human-merge pairs (the 50 human-validated pairs are excluded). The LLM judge is developed to output a binary label (*Direct fix* vs. *Non-fix*) using the 45 agreed pairs during human validation. After the labeling process, the judge’s result is re-validated with the second author relabeling, over newly sampled pairs of agent PRs ($n=50$) and human PRs ($n=50$). Eventually, the LLM judge substantially matches the human at $\kappa = 0.78$ in agent pairs, and its *Direct-fix* precision is $27/30 = 90\%$ on both cohorts (pooled $54/60 = 90\%$, Wilson 95% CI = 80–95%).

4.1.2 Results of RQ1

Table 1 (RQ1) The candidate, verified, and precision-adjusted rates on a full agent cohort.

PR <i>P</i> Author	PRs (<i>n</i>)	Candidate (%)	Verified (%)	Adjusted (%)
OpenAI Codex	2,049	24.8	5.5	4.9 (4.4–5.2)
Cursor	153	15.7	5.2	5.0 (4.8–5.1)
Devin	1,519	21.5	3.7	3.4 (3.0–3.5)
Claude Code	63	4.8	3.2	3.2
GitHub Copilot	721	23.4	3.5	3.1 (2.8–3.3)
AI Agent Overall	4,505	22.9	4.5	4.1 (3.7–4.3)

Approximately 4 % of agent merges are followed by a verified fix.

Table 1 shows the percentages of agent PRs that are followed by candidate or verified fix, including the precision adjusted rate. Among the 4,505 merged agent PRs with a fully observed 30-day window, 22.9% are followed by a candidate fix ($n=1,032$) and 4.5% are verified fix ($n=203$). The rate ranges from 3.2% (Claude Code, $n=63$) to 5.5% (OpenAI Codex, $n=2,049$). It is worth noting that the population of Claude Code is small due to half of the PRs (67 of 130) being merged after the 30-day cut-off date, so they are dropped. Specifically, 203 of the 4,505 PRs carry a verified fix, including 189 judge-verified and 14 human-verified consensus on the gold pairs.

Table 2 (RQ1) The cross-cohort comparison on the shared observation window (both cohorts cut at June 28, 2025).

Cohort	PRs (<i>n</i>)	Candidate (%)	Verified (%)	Adjusted (%)
AI agents	2,012	20.9	3.68	3.34 (3.00–3.52)
Humans	4,063	14.7	2.34	2.10 (1.87–2.23)
Odds ratio, within repositories (218 strata)		1.41 [1.19–1.68]	1.62 [1.10–2.39]	—

Agent merges attract verified fixes at 1.62 times the human odds.

In the shared observation window, 3.68 % of agent merges (74 of 2,012) attract a verified fix against 2.34 % of human merges (95 of 4,063). Compared only within the same repositories, the gap holds: the Mantel–Haenszel odds ratio (Mantel and Haenszel, 1959), stratified over the 218 repositories where both cohorts appear, is 1.62 (95 % CI=1.10–2.39, $p=0.015$). The Mantel–Haenszel risk ratio (Greenland and Robins, 1985) is also 1.59. Therefore, over the same period of time, an agent merge is roughly 1.6 times as likely as a human merge in the same repository to be followed by a verified fix.

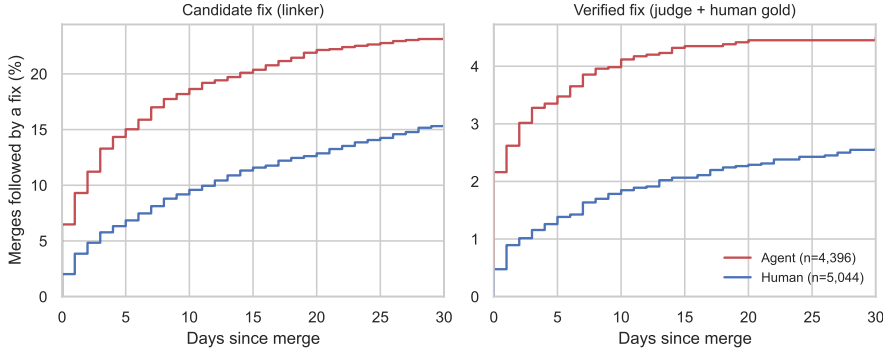


Fig. 3 (RQ1) Cumulative incidence of the first follow-up fix after the merge, agent vs. human cohorts on the shared observation window (every merge enters with its own time).

The fixes arrive early, and the gap is present from the first day. We report Kaplan-Meier cumulative incidence curves (Kaplan and Meier, 1958) in Figure 3. The time-to-event view shows every merge up to the shared cut-off with its own observation time. The curves are separate from the first day, showing that agent PRs receive more fixes than human PRs under both the candidate and verified fix definitions. In fact, half of the 30-day incidence accumulates within the first week in both agent and human cohorts, and the cumulative incidence reaches 4.5 % vs. 2.6 % by day 30 on the verified fix.

Takeaway RQ1: 22.9 % of merged agent PRs are followed by a candidate fix and 4.5 % by a verified fix within 30 days. In the same repositories and the same observation window, agent merges are fixed at 1.41 times the human odds (candidate) and 1.62 times (verified), and the fixes arrive early: half of the 30-day incidence accumulates within the first week.

4.2 RQ2: Who authors the verified fix PRs of the merged agent PRs?

In this research question, our goal is to track the author of the verified fix (PR Q), whether it is from the agent itself, another agent, or a human.

4.2.1 Approach of RQ2

For all *verified* pairs, we classify the author by the identity that opened the fixing PR Q . The author is classified into three types: **human**, **self-agent**, and **other-agent** by the following criteria:

- **human**: when Q comes from human
- **self-agent**: when Q is an agent PR opened by the same agent as P
- **other-agent**: otherwise, including bots

We report the split over verified fixes. The same classification is applied to the verified pairs of human cohort as a baseline on who fixes human merges.

4.2.2 Results of RQ2

Table 3 (RQ2) The author of the verified fix, i.e., PR Q .

PR P Author	Pair (n)	Self-agent (%)	Human (%)	Other agent (%)
OpenAI Codex	125	59.2	38.4	2.4
Devin	88	72.7	23.9	3.4
GitHub Copilot	40	95.0	5.0	0.0
Cursor	8	87.5	0.0	12.5
Claude Code	2	0.0	50.0	50.0
AI Agent Overall	263	69.6	27.4	3.0
Human	110	—	89.1	10.9

Agents largely fix their own merges by 69.6 %. Table 3 shows the authors of verified fix (PR Q). Classifying the fixer of every verified agent-merge pair by who opened the fixing PR Q : 69.6 % are *self*-fixes, meaning that the same agent that shipped the merge also ships the fix, against 27.4 % fixed by a human and 3.0 % by a different agent ($n=263$ verified fixes). The pattern holds for every agent with a non-trivial count. Codex is the agent whose merges humans most often fix (38.4 %), while Copilot self-fixes about 95 % of its merges. Note that Claude Code’s population is small from RQ1, because most PRs fall after the 30-day cut-off date.

Human merges are also fixed by humans for 89.1 %. The baseline analysis of the verified fixes of the human cohort (Table 3, Human row) also shows that 89.1 % are fixed by humans and 10.9 % by an agent ($n=110$). Fixing work thus stays largely within the authoring population on both sides of the comparison. Agents fix their own merges, and humans fix theirs.

Takeaway RQ2: 69.6 % of the verified fixes on agent merges come from the same agent, while 89 % of the verified fixes on human merges also come from humans. The finding shows that fixing work stays within the authoring population on both sides.

4.3 RQ3: To what extent are the commits pushed to the verified fix PRs authored by agents?

In this research question, we zoom into the commit-level of PR Q to answer who actually authors those commits in the verified fix PRs.

4.3.1 Approach of RQ3

We classify every commit on each merged PR as **agent**, **automation**, or **human**, extending the rule-based comment-classifier of Duma et al. (2026) (96.5% accuracy on their 800-comment validation sample). Specifically, we apply two main rules:

1. The *author login* matches agent or bot account patterns (see Appendix B).
2. The *commit message* matches agent-generated message (see Appendix B).

Note that OpenAI Codex does not sign its commits (only PR-level). In fact, they are authored under the developer’s GitHub login, and fewer than 2% carry any marker. Therefore, Codex’s split is excluded from the commit measure and we report only the overall commit volume. It is also worth noting that because an agent commit that lacks both marker and agent login is counted as **human**, the agent-authored share in this work can be considered a conservative *lower bound*.

In the analysis, we aggregate results *per PR*, not over the aggregated commit stream. Thus, each merged PR contributes its own agent-authored share, and we report two numbers: the average of those shares per PR as *Avg Agent Share per PR* and the percentage of PRs whose commits are all agent-authored as *PR/w All Commits by Agent*. We also run the same classification over the verified fix of the human cohort for a baseline comparison.

4.3.2 Results of RQ3

Table 4 (RQ3) Commit author-class inside the verified fix PRs, aggregated per PR.

PR Q Author	PRs (n)	Commits median (n)	Avg Agent Share per PR (%)	PR/w All Commits by Agent (%)
GitHub Copilot	40	3.0	86.6	70.0
Devin	59	1.0	92.0	84.7
Cursor	23	1.0	79.1	69.6
Claude Code	1	8.0	—	—
OpenAI Codex	103	1.0	—	—
All (excl. Codex)	123	2.0	87.4	76.4
Human	124	2.0	0.8	0.8

The commits added on a verified fix of agent PR are majority agent-authored with the average share of 87.4%. Classifying every

commit of the unique verified fix PRs on marker and login evidence (Table 4), the result shows that agent-opened fix PRs are 79–92 % agent-authored per PR, and 76.4 % of those PRs are agent-authored throughout all commits. On human-opened fix PRs, the same classifier carries almost no marked agent work (0.8 % average per-PR share over 124 PRs and 554 commits). Therefore, who opens the fix PR is, on marked evidence, also who writes it.

Table 5 Commit author-class on *all* merged AIDev-pop PRs. OpenAI Codex is unmeasurable (—) as it does not sign its commits.

PR Author	PRs (<i>n</i>)	Commits median (<i>n</i>)	Avg Agent Share per PR (%)	PR/w All Commits by Agent (%)
GitHub Copilot	1,428	4.0	81.6	51.8
Devin	1,813	3.0	80.1	56.9
Cursor	569	2.0	72.9	54.1
Claude Code	130	2.5	57.1	40.0
OpenAI Codex	2,834	1.0	—	—
All (excl. Codex)	3,940	3.0	78.8	54.1
Human	4,900	2.0	0.9	0.6

AI Agents write the fixes they open, more often than they write their merges. We zoom out to the author-class on all merged PRs in AIDev-pop (Table 5). Across the full 3,940 PRs that include 123 fix PRs, the average agent-authored share is 78.8 % overall across four AI agents (exclude Codex), and 54.1 % of those PRs are all agent-authored commits. This is below the 87.4 % and 76.4 % the same agents record on only their verified fix PRs. Therefore, this result indicates that agents have a higher share of commits on fix PRs than on their merged PRs in general.

Takeaway RQ3: The commits added to verified-fix agent PRs between first push and merge are, on average, 87.4 % agent-authored per PR, and 76.4 % of those PRs are agent-authored throughout all commits.

4.4 RQ4: How do merge-time signals differ between merged agent PRs that are followed by a fix and those that are not?

In this research question, our goal is to find the characteristics that differ between agent PRs that are followed by a fix and those that are not.

4.4.1 Approach

To compare the distributions, we use four merge-time signals of agent merges: time under review, commits per PR, post-first-push churn, and review items,

shown as split violins. For each signal we test the two-group difference with a Mann–Whitney U test (Mann and Whitney, 1947) and measure its magnitude with Cliff’s δ (Cliff, 1993; Romano et al., 2006) against the magnitude labels per Romano et al. (2006).

Additionally, because pooled contrasts can reflect repository confounds rather than merge properties, each signal is also tested *within* repositories with a conditional logistic regression (Chamberlain, 1980), one model per signal:

$$\text{logit } P(Y_{ri} = 1) = \alpha_r + \beta \log_{10}(1 + s_{ri}) + \gamma \log_{10}(1 + f_{ri}), \quad (1)$$

where Y_{ri} indicates that merge i in repository r is followed by a verified fix within 30 days; s_{ri} is the merge-time signal; f_{ri} is the number of non-boilerplate files shipped (i.e., PR size control); and α_r is a repository control. The reported odds ratio e^β compares two same-repository merges ten times apart on each signal due to a log-scale (Figure 5).

4.4.2 Results

Figure 4 shows the four signal distributions of the two groups side by side, and Table 6 shows the significance tests and their effect sizes.

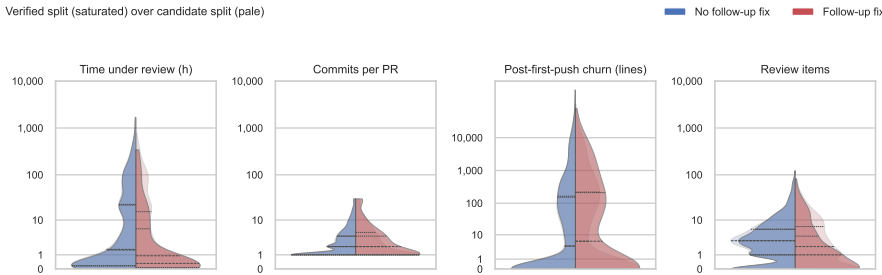


Fig. 4 (RQ4) The signal distributions of agent PRs with and without a follow-up fix.

Table 6 (RQ4) Per-signal comparison of agent PRs with and without a follow-up fix. A negative δ means the w/ follow-up fix merges sit *lower* on the signal.

Signal	Candidate			Verified		
	p	δ	Effect size	p	δ	Effect size
Time under review	<0.001	-0.100	(negl.)	<0.001	-0.221	(small)
Commits per PR	0.15	+0.028	(negl.)	0.75	+0.013	(negl.)
Post-first-push churn	0.21	+0.024	(negl.)	0.41	+0.032	(negl.)
Review items	<0.001	-0.101	(negl.)	<0.001	-0.239	(small)

In the pooled comparison, the two groups are nearly identical, with the follow-up fix PRs group having slightly less time under

review and fewer review items (small effects). Our four signals show that the difference in effect size is negligible to small (Table 6). The two differences that reach the significance point are in *negative* effect sizes. Specifically, the merged agent PRs with a follow-up fix yield slightly shorter review time and fewer review items than the group without a follow-up fix. The result indicates that agent PRs with a lower code review (time and items) correlate with slightly more follow-up fixes in the pooled view.

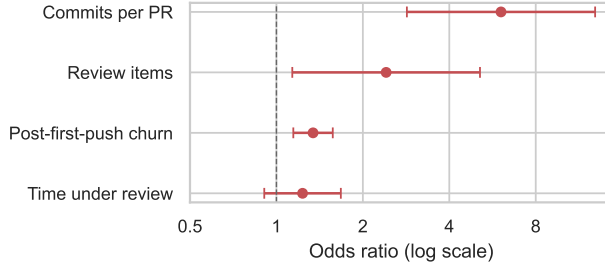


Fig. 5 Within-repository, odds of a verified follow-up fix, per signal. Each dot is the odds ratio comparing merges ten times apart on a signal, with a 95 % confidence interval.

In the within-repositories comparison, the effort to complete a PR could mark a follow-up fix. When the repository and the number of shipped files are held fixed, the result changes (Figure 5). Our finding shows that when a PR has ten times the commits of the other, the one with more commits has 6.1 times the odds of being followed by a verified fix ($p < 0.001$). On the same tenfold scale, review items carry 2.4 times ($p = 0.022$), churn carry 1.3 times ($p < 0.001$), and time under review shows no significant effect ($p = 0.18$). Therefore, the result shows that the PRs that needed many rounds of work before the merge are the ones most likely to need more work after it, when compared within the repository.

Takeaway RQ4: Overall, merge-time signal distributions of agent PRs on both groups are nearly identical (all effects negligible to small). However, within repositories, the number of commits to complete a PR could mark a follow-up fix. Specifically, a merge with ten times the commits carries 6.1 times the odds of a verified fix.

5 Discussion

In this section, we discuss our findings across research questions.

5.1 The merge is not the end of the work, and code review is needed.

Previous studies of agent PRs largely stop at the merge (e.g., size, review activity, and acceptance (Li et al., 2025; Ogenrwot and Businge, 2026; Duma et al., 2026)) as a finished work. Our results show that the work continues, at a small but consistent rate: 4.5% of agent merges carry a verified fix within 30 days, which is 1.62 times more than human merges (RQ1). However, our RQ4 findings also show that, at the aggregated level, the signals at the merge time between PRs with and without a follow-up fix have only minor differences. Therefore, the study indicates that a PR should also be monitored beyond the merge, and the signal may live in the code rather than in the merge metadata. In fact, on AIDev PRs, a content-level model (i.e., code semantics and lexicon) that reads the code changes can predict the post-merge modification at AUC=0.671 (Rahman and Shihab, 2026). Therefore, in line with our study, this indicates that verification of agent contributions requires code review (i.e., reading the code changes). *Future research* should measure how much of the agent fix that code review can capture and explore a monitoring framework for the agent platform to catch issues before and after a merge.

5.2 Owners fix their own code, but now the owner is an agent.

Code ownership has been a long-standing focus in software engineering for many years. Code changes are recommended to be made by an owner rather than by outsiders for less defects (Bird et al., 2011; Greiler et al., 2015). Before the age of AI coding agents, humans fix roughly half of their own defects at individual granularity (Zhu and Godfrey, 2021; Keller et al., 2023; Tan et al., 2022). This is in line with our findings (RQ2 and RQ3) that human merges are still fixed by humans 89% of the time. Similarly to humans, our results indicate that agent merges are also fixed by the same agent 69.6% of the time. So, our findings show the same pattern of self-fixing in both agent and human cohorts, similar to the history, but with the new *operational* owner. This partially addresses the concern that the AI agent code goes unmaintained (Sawada et al., 2026), at least at the fix level. In human teams, abandoned code is adopted by experienced insiders (Rigby et al., 2016), and that adoption depends on someone having accumulated the experience while the owner was active; an agent that authors a merge and then works on its own fixes leaves no such insider behind. Therefore, the next question is about the deeper granularity of what happens to the agent code after the agent *model* that owns the merge is deprecated or discontinued, and whether projects should adopt explicit ownership policies similar to those for human. *Future research* should explore the model granularity and how the model and agent ownership affects the codebase.

6 Threats to Validity

Construct Validity. Author-class counts measure who *committed or marked* a change, not who wrote it, and OpenAI Codex’s commits carry no marker. We mitigate this by reporting the agent-authored shares (RQ3) as floors and excluding Codex from the commit decomposition. The *candidate fix* label links by file co-location within 30 days, so it can miss cross-file fixes, fixes slower than 30 days, and fixes folded into non-`fix`-typed PRs. We mitigate this by applying identical linking criteria to both cohorts, so the cross-cohort comparison holds, and by reading the verified rates as floors. Commit fetches cap at 30 per PR (AIDev’s cap); under 2 % of PRs reach it, so we accept the truncation.

Internal Validity. Agent labels in AIDev derive from commit-author patterns, so an unmarked agent PR hiding in the *human* table would count as a human author. This error can inflate the human-fixed share; however, the results show that our findings remain the same for agents consistently fixing more than half of their code. Additionally, fixer identity captures the authorship side of ownership only; we acknowledge that ownership can also be held through code review (Thongtanunam et al., 2016) and strictly scope the claim to only who authors the fix.

External Validity. Our population is the five AIDev agents in popular (≥ 500 -star) repositories, so other agents, low-activity repositories, and behaviour after the data window are out of scope; we state the findings for this population rather than for agent PRs in general. Our snapshot admits more repositories than Li et al. (2026)’s (1,479 vs. 856 at ≥ 500 stars) because stars and PRs accrue over time. We verified that the snapshot reproduces their ≥ 101 -star curated set exactly (2,807 repositories), so the difference is accrual, not a divergent population.

Conclusion Validity. Verification is reliable at the binary Direct-fix level but fluctuate at three-way labels (human–human $\kappa=0.77$ binary vs. 0.60 three-way; judge–human 0.78 vs. 0.46). We mitigate this by using the binary construct for every headline claim. An LLM judge could favour one cohort, so we checked this with blind spot-checks on both sides: its Direct-fix precision is $27/30 = 90\%$ on both agent-merge and human-merge pairs. Thus, the adjusted rates scale by the pooled estimate ($54/60 = 90\%$, Wilson 80–95 %). Pooled cross-cohort rates would mix time and repository effects. We mitigate this by comparing on the shared observation window with repository-stratified (Mantel–Haenszel) estimates. The single-cohort rates keep each cohort’s own window because the shared cut-off shrinks the agent cohort unevenly (e.g., leaving Cursor with four observable merges and GitHub Copilot with no verified fix); we therefore never compare cohorts on the own-window rates.

7 Conclusion

A merged AI-agent pull request is usually counted as finished work, but previous studies show that agent work can leave issues behind. In this paper, we ask how often an agent PR is followed by a fix and who actually finishes it. We tracked 6,774 merged agent PRs in AIDev-pop into their follow-up fixes, compared to a human baseline in the same repositories. We found that agent merges attracted verified fixes at 1.62 times the odds of human merges over the same observation window. Of the verified fixes in agent merges, 69.6% come from the same agent. The commits to verified fix PRs of agent merges were substantially agent-authored (on average 87.4% per PR). The merges that later needed a fix have nearly the same signals as the others at merge time; the clearest signal, within repositories, was how many commits the merge itself had taken. Agents largely finish their own job on both sides of the merge, but their merges need fixing modestly more often than human merges, with little visibility up front. As agents author more of the code that ships, following their work past the merge is what will keep evaluation aligned with how software is now produced.

Statements and Declarations

Funding: This work is supported by the Australian Composites Manufacturing CRC under Grant Number: HPN003 – ForgeX.ai, AI factory management (Phase 1).

Author Contributions: The conception and design of the study, data collection and data analysis were carried out by Wannita Takerngsaksiri. Materials preparation and human validation were performed by Wannita Takerngsaksiri and Nhat Duong. The first draft of the manuscript was written by Wannita Takerngsaksiri. The review and acquisition of funding was done by Scott Barnett. All authors read, provided comments, and approved the final manuscript.

Data Availability Statement: The replication package is available at <https://github.com/wannita901/pr-fix-authorship>. The pull requests are drawn from the public AIDev dataset, mined from public GitHub repositories; no private or personally identifiable data are used.

Conflict of Interest: The author declares no conflict of interest.

Ethical approval: Not applicable.

Informed consent: Not applicable.

Clinical Trial Number: Not applicable.

References

- Alam K, Mondal S, Roy B (2026) Why are AI agent involved pull requests (fix-related) remain unmerged? an empirical study. URL <https://arxiv.org/abs/2602.00164>, 2602.00164
- Anthropic (2026) Claude Code by anthropic — ai coding agent, terminal, ide. URL <https://claude.com/product/claude-code>
- AnySphere (2026) Cursor: Ai coding agent for building ambitious software. URL <https://cursor.com>
- Bird C, Nagappan N, Murphy B, Gall H, Devanbu P (2011) Don't touch my code! Examining the effects of ownership on software quality. In: Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering (ESEC/FSE), pp 4–14, DOI 10.1145/2025113.2025119
- Chamberlain G (1980) Analysis of covariance with qualitative data. The Review of Economic Studies 47(1):225–238, DOI 10.2307/2297110
- Cliff N (1993) Dominance statistics: Ordinal analyses to answer ordinal questions. Psychological Bulletin 114(3):494–509
- Cognition (2026) Devin, the ai software engineer. URL <https://devin.ai>
- Cohen J (1960) A coefficient of agreement for nominal scales. Educational and Psychological Measurement 20(1):37–46, DOI 10.1177/001316446002000104
- Duma K, Wróblewski P, Bobińska J, Winiarska J, Przymus P (2026) These aren't the reviews you're looking for: How humans review AI-generated pull requests. In: Proceedings of the International Conference on Evaluation and Assessment in Software Engineering (EASE), arXiv:2605.02273
- Foucault M, Teyton C, Lo D, Blanc X, Falleri JR (2015) On the usefulness of ownership metrics in open-source software projects. Information and Software Technology 64:102–112, DOI 10.1016/j.infsof.2015.01.013
- GitHub (2026) GitHub Copilot, your ai pair programmer. URL <https://github.com/features/copilot>
- Gong J, Pinna G, Bian Y, Zhang JM (2026) Analyzing message-code inconsistency in AI coding agent-authored pull requests. In: Proceedings of the 23rd International Conference on Mining Software Repositories (MSR 2026), Mining Challenge, DOI 10.1145/3793302.3793583
- Greenland S, Robins JM (1985) Estimation of a common effect parameter from sparse follow-up data. Biometrics 41(1):55–68, DOI 10.2307/2530643
- Greiler M, Herzig K, Czerwinka J (2015) Code ownership and software quality: A replication study. In: Proceedings of the 12th IEEE/ACM Working Conference on Mining Software Repositories (MSR), pp 2–12, DOI 10.1109/MSR.2015.8
- Haider MA, Zimmermann T (2026) Understanding dominant themes in reviewing agentic AI-authored code. In: Proceedings of the 23rd International Conference on Mining Software Repositories (MSR 2026), Mining Challenge, DOI 10.1145/3793302.3793566
- Kaplan EL, Meier P (1958) Nonparametric estimation from incomplete observations. Journal of the American Statistical Association 53(282):457–481,

- DOI 10.1080/01621459.1958.10501452
- Keller B, Meneely A, Meyers BS (2023) What happens when we fuzz? Investigating OSS-Fuzz bug history. In: Proceedings of the 20th IEEE/ACM International Conference on Mining Software Repositories (MSR), pp 198–210, DOI 10.1109/MSR59073.2023.00038
- Khemissi I, Chouchen M, Wang D, Kula RG (2026) Humans integrate, agents fix: How agent-authored pull requests are referenced in practice. In: Proceedings of the 23rd International Conference on Mining Software Repositories (MSR 2026), Mining Challenge, DOI 10.1145/3793302.3793595
- Landis JR, Koch GG (1977) The measurement of observer agreement for categorical data. *Biometrics* 33(1):159–174
- Li H, Zhang H, Hassan AE (2025) The rise of AI teammates in software engineering (SE) 3.0: How autonomous coding agents are reshaping software engineering. URL <https://arxiv.org/abs/2507.15003>, 2507.15003
- Li H, Zhang H, Hassan AE (2026) AIDev: Studying AI coding agents on GitHub. URL <https://arxiv.org/abs/2602.09185>, 2602.09185
- Liu Y, Widayarsi R, Zhao Y, Irsan IC, Chen J, Lo D (2026) Debt behind the AI boom: A large-scale empirical study of AI-generated code in the wild. URL <https://arxiv.org/abs/2603.28592>, 2603.28592
- Mann HB, Whitney DR (1947) On a test of whether one of two random variables is stochastically larger than the other. *The Annals of Mathematical Statistics* 18(1):50–60, DOI 10.1214/aoms/1177730491
- Mantel N, Haenszel W (1959) Statistical aspects of the analysis of data from retrospective studies of disease. *Journal of the National Cancer Institute* 22(4):719–748, DOI 10.1093/jnci/22.4.719
- Ogenrwot D, Businge J (2026) How AI coding agents modify code: A large-scale study of GitHub pull requests. In: Proceedings of the 23rd IEEE/ACM International Conference on Mining Software Repositories (MSR)—Mining Challenge Track, arXiv:2601.17581
- OpenAI (2026) Codex the same powerful coding agent—now in chatgpt. URL <https://openai.com/codex/>
- Peralta SRO, Hoshi F, Washizaki H, Ubayashi N, Kondo I, Higo Y, Mukai H, Yoshida N, Kusama K, Tanaka H, et al. (2026) Why are agentic pull requests merged or rejected? an empirical study. In: Proceedings of the 23rd International Conference on Mining Software Repositories, pp 787–791
- Rahman M, Shihab E (2026) Will it survive? deciphering the fate of AI-generated code in open source. URL <https://arxiv.org/abs/2601.16809>, to appear at EASE 2026, 2601.16809
- Rigby PC, Zhu YC, Donadelli SM, Mockus A (2016) Quantifying and mitigating turnover-induced knowledge loss: Case studies of Chrome and a project at Avaya. In: Proceedings of the 38th International Conference on Software Engineering (ICSE), pp 1006–1016, DOI 10.1145/2884781.2884851
- Robbes R, Matricon T, Degueule T, Hora A, Zacchiroli S (2026) Agentic much? adoption of coding agents on github. *ACM Trans Softw Eng Methodol* DOI 10.1145/3822180, URL <https://doi.org/10.1145/3822180>, just Accepted

- Romano J, Kromrey JD, Coraggio J, Skowronek J (2006) Appropriate statistics for ordinal level data: Should we really be using t-test and cohen's d for evaluating group differences on the nsse and other surveys? In: Annual Meeting of the Florida Association of Institutional Research, pp 1–33
- Sawada S, Shirai T, Kashiwa Y, Yamaguchi K, Iwata H, Iida H (2026) To what extent does agent-generated code require maintenance? An empirical study. In: Proceedings of the 30th International Conference on Evaluation and Assessment in Software Engineering (EASE), arXiv:2605.06464
- Seo J, Deldari S, Mentis HM (2026) Whose code is it? How AI autonomy reshapes ownership, responsibility, and disclosure in AI-assisted programming. In: Proceedings of the 31st International Conference on Intelligent User Interfaces (IUI), DOI 10.1145/3742413.3789121
- Takerngsaksiri W, Pasuksmit J, Thongtanunam P, Tantithamthavorn C, Zhang R, Jiang F, Li J, Cook E, Chen K, Wu M (2025) Human-in-the-loop software development agents. In: 2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP), IEEE, pp 342–352
- Tan J, Feitosa D, Avgeriou P (2021) Do practitioners intentionally self-fix technical debt and why? In: Proceedings of the 37th IEEE International Conference on Software Maintenance and Evolution (ICSME), pp 251–262, DOI 10.1109/ICSME52107.2021.00029
- Tan J, Feitosa D, Avgeriou P (2022) Does it matter who pays back technical debt? An empirical study of self-fixed TD. *Information and Software Technology* 143:106738, DOI 10.1016/j.infsof.2021.106738
- Thongtanunam P, McIntosh S, Hassan AE, Iida H (2016) Revisiting code ownership and its relationship with software quality in the scope of modern code review. In: Proceedings of the 38th International Conference on Software Engineering (ICSE), pp 1039–1050, DOI 10.1145/2884781.2884852
- Treude C (2026) Accountable agents in software engineering: An analysis of terms of service and a research roadmap. In: Proceedings of the 3rd ACM International Conference on AI-Powered Software (AIware), DOI 10.1145/3805760.3814889
- Watanabe K, Tsuchida R, Monno T, Huang B, Yamasaki K, Fan Y, Shimari K, Matsumoto K (2026) How AI coding agents communicate: A study of pull request description characteristics and human review responses. URL <https://arxiv.org/abs/2602.17084>, 2602.17084
- Zhu W, Godfrey MW (2021) Mea culpa: How developers fix their own simple bugs differently from other developers. In: Proceedings of the 18th IEEE/ACM International Conference on Mining Software Repositories (MSR), pp 515–519, DOI 10.1109/MSR52588.2021.00065

A The Labeler for Verified Fix Annotation

Figure 6 shows a web labeler screen that human annotators use to label the candidate pair: Original $P \rightarrow$ Candidate Fix Q (§4.1.1). Each pair is labeled with one of the four labels defined below:

- *Direct fix*: Q exists to *repair* a defect, regression, or wrong/incomplete behavior that P introduced or left behind; Q 's modifies or reverts the lines P added in the shared files.
- *Related touch*: Q does *incremental or additive work* in the same area as P (e.g., extending or refactoring it) without repairing a defect.
- *Unrelated/mislabeled*: Q touches the shared file for an *independent reason*.
- *Indeterminate*: the material is *insufficient* to decide.

We consider a pair a *verified fix* only when labeled *Direct fix*.

#8 Cursor gap 7d Q: fix 2 subst · 0 boiler (net)

P (original): <https://github.com/keephq/keep/pull/5002> [Files ·]
feat: Improve selected columns storage mechanism

Q (candidate fix): <https://github.com/keephq/keep/pull/5113> [Files ·]
fix: bug with saving presets and default columns

SUBSTANTIVE SHARED FILES (2)

- ▶ keep-ui/widgets/alerts-table/ui/ColumnSelection.tsx [Q diff] [P diff]
- ▶ pyproject.toml [Q diff] [P diff]

☒ Direct fix
 ☐ Related touch
 ☐ Unrelated/mislabeled
 ☐ Indeterminate

reason (≤15 words)

Fig. 6 A sample screen of web labeler for human to annotate a verified fix.

B The Rule List of Commit Author-Class Mapping (RQ3)

There are two main rules to map the evidence of author-class in RQ3 (§4.3.1): *author login* and *commit message*. Table 7 is the complete set of commit author-class rules.

Table 7 The author-class evidence rules per agent.

Class	Author Login	Commit Message
GitHub Copilot	Copilot	Co-Authored-By: Copilot
Devin	devin-ai-integration[bot]	Devin Run Link; Co-Authored-By: Devin
Cursor	cursoragent	Co-Authored-By: Cursor
Claude Code	claude[bot], claude-code	Generated with + robot emoji; Generated with [Claude Code; Co-Authored-By: Claude
OpenAI Codex	codex, codex-cli	Generated with ... Codex; Co-Authored-By: openai-codex
automation	any other *[bot]; dependabot, renovate, codecov, github-actions, autofix-ci, vercel, netlify, semantic-release-bot, pre-commit-ci, allcontributors	—
human	everything else (default)	—